# DESIGN PATTERN MAPPING:

## 1. Concurrency & High-Throughput Patterns

- Singleton Pattern (Redis Lua Script Engine):
  - Problem: Race conditions and database row-locking when 10,000+ users hit stock at the same millisecond.
  - Solution: Maintains a single, globally initialized Redis Lua script instance executing atomically in single-threaded memory (EVALSHA). Ensures zero overselling without distributed locks.
- Observer Pattern (Keyspace TTL Expiry Watcher):
  - Problem: Stock locked by non-paying users remains blocked indefinitely.
  - Solution: The Inventory Service subscribes to Redis __keyevent@0__:expired events. When a 10-minute reservation TTL expires, it automatically triggers a stock rollback to the available pool.
- Idempotency Key Pattern (Request Deduplication):
  - Problem: Network lag causing users to aggressively double-tap "Buy Now", leading to duplicate orders or billing.
  - Solution: A unique Idempotency-Key (UUID) generated on the client side is verified via Redis fast-locks before executing write handlers. Duplicate requests return cached responses.

## 2. Microservice Orchestration & Workflow Patterns

- Saga Orchestrator Pattern (Distributed Transactions):
  - Problem: Maintaining data consistency across decoupled services (Inventory → Order → Payment) without heavy 2-Phase Commit (2PC) locks.
  - Solution: An OrderSagaOrchestrator manages forward transactions (ReserveStock → ProcessPayment → ConfirmOrder) and executes compensating actions (ReleaseStock, CancelOrder) if payment fails.
- State Pattern (Order Lifecycle Management):
  - Problem: Invalid order state mutations (e.g., fulfilling an order before payment confirmation).
  - Solution: Encapsulates state rules in explicit state handlers (PendingState, ReservedState, PaidState, ExpiredState). State transitions are strictly controlled by domain events.
- Transactional Outbox Pattern (Reliable Event Dispatch):
  - Problem: Loss of event messages if the application crashes immediately after writing to the database but before publishing to Kafka.
  - Solution: Writes domain events directly to an outbox table inside the same local database transaction. A background CDC engine (Debezium/Poller) reads and streams events to Kafka reliably.

## 3. Structural, Behavioral & Resilience Patterns

- Strategy & Factory Patterns (Payment Gateway Abstraction):
    - Problem: Vendor lock-in and handling distinct payload structures for Stripe, Razorpay, or UPI.
    - Solution: A PaymentGatewayFactory instantiates the appropriate adapter (RazorpayStrategy, StripeStrategy) at runtime through a common PaymentGateway interface.
- Circuit Breaker Pattern (Fault Tolerance & Degradation):
    - Problem: Slow external dependencies (e.g., Payment Gateway timeouts) cascading to consume application threads.
    - Solution: Implements Resilience4j / Envoy circuit breakers. If gateway failure rates exceed 50%, the breaker opens immediately, falling back to queued processing or graceful rejection.
- Adapter / Proxy Pattern (API Gateway Rate-Limiting & Auth):
    - Problem: Invalid traffic, bots, and unauthenticated requests reaching downstream microservices.
    - Solution: API Gateway acts as a proxy executing Token-Bucket rate limiting and JWT verification at ingress, protecting backend services from traffic bursts.